FUTURE DATA LAB

DSA 1080A | Programming for Data Science | Five-week group project

PROJECT AT A GLANCE	REQUIREMENT
Level	Beginner Python, Pandas and Matplotlib
Teams	4–5 students; select ONE project
Dataset	At least 15,000 rows × 10 meaningful columns
Duration	5 consecutive weeks with graded weekly checkpoints
Submission	GitHub repository URL through Blackboard; final presentation
Total	100 marks

THE BRIEF

Build a reproducible data investigation about a future-facing problem. Your team will obtain a dataset, validate its scale, explore and clean it in Pandas, answer evidence-based questions, create Matplotlib visualizations and communicate a realistic recommendation. This is a data analysis project—not an AI model, mobile app, or advanced software engineering project.

Mandatory rules

- Every team must use one coherent dataset containing at least 15,000 data rows (header excluded) and at least 10 meaningful columns. A row-count screenshot alone is not sufficient; validate shape using Python and document it.
- If the original dataset contains multiple files, a joined/concatenated analysis dataset must meet the minimum. Do not inflate a small dataset by copying/repeating rows or inventing fake transactions.
- Use real, licensed/open or lawfully collected and anonymized data. If a synthetic dataset is approved, label it clearly, explain generation assumptions and avoid presenting invented patterns as real-world evidence.
- Stay within Python fundamentals, Pandas and Matplotlib; Seaborn, Plotly, machine learning, APIs and dashboards are optional but earn no extra marks and cannot replace required work.
- Each member should contribute commits and explain their own work; submit the same repository URL at every checkpoint, not separate copies.

1 | Choose one of ten future-facing investigations

01 AI Learning Assistant Adoption

Examine how students interact with AI-powered study tools without evaluating or building an AI model.

Suggested 10-column schema: session_id, date, course, study_tool, task_type, minutes_used, prompt_count, satisfaction_score, completion_status, institution_type.

Example questions: Which tasks are most common? How do study duration and satisfaction differ by tool or course? Which weeks have the highest usage?

02 Electric Mobility and Charging Patterns

Explore charging events and electricity demand in an imagined future electric-vehicle ecosystem using authentic public EV/charging data.

Suggested 10-column schema: session_id, start_time, station_id, location, vehicle_type, connector_type, energy_kwh, duration_min, cost, day_type.

Example questions: Where and when is charging busiest? Which station types have longer sessions? How does charging cost vary with energy supplied?

03 Smart Agriculture and Climate Resilience

Investigate crop/environment observations and practical indicators of farm performance.

Suggested 10-column schema: record_id, observation_date, region, crop, rainfall_mm, temperature_c, humidity_pct, soil_moisture, yield_kg, plot_area_ha.

Example questions: Which regions or crops have higher yields? How does rainfall vary by region? What differences emerge across growing periods?

04 Cybersecurity Threat Event Trends

Investigate de-identified security logs to understand frequency, severity and timing of incidents.

Suggested 10-column schema: event_id, timestamp, source_region, event_type, severity, outcome, device_type, protocol, hour, response_minutes.

Example questions: Which event categories dominate? Do severe events cluster by hour or device? How does response time vary across incident types?

05 Future City Air Quality Sensors

Analyze public sensor records showing urban pollution and environmental conditions.

Suggested 10-column schema: observation_id, timestamp, station_id, district, pm25, pm10, no2, temperature_c, humidity_pct, air_quality_category.

Example questions: Which districts have higher particulate levels? Which periods have the worst readings? Are patterns consistent across sensors?

06 Digital Wallet Transactions and Consumer Habits

Analyze anonymized digital payment transactions to understand spending categories and usage trends.

Suggested 10-column schema: transaction_id, timestamp, merchant_category, amount, currency, payment_channel, customer_segment, region, transaction_status, day_of_week.

Example questions: What categories account for most spending? How does transaction value change across periods? Which payment channels see more failures?

07 Renewable Energy and Smart Grid Demand

Investigate renewable generation and consumption in a modern electricity network.

Suggested 10-column schema: record_id, timestamp, site, energy_source, generation_kwh, demand_kwh, temperature_c, wind_speed, solar_irradiance, grid_status.

Example questions: When does demand exceed generation? Which sources contribute most? What changes are visible by hour or month?

08 Smart Public Transport and Urban Commutes

Study transit ridership and delays using public transport logs or GTFS-derived observations.

Suggested 10-column schema: trip_id, service_date, route_id, origin_zone, destination_zone, planned_time, actual_time, delay_min, passenger_count, fare.

Example questions: Which routes have the most delays? When is ridership highest? Which days show the biggest variation in trip performance?

09 E-commerce of the Future: Sustainable Shopping

Investigate online order patterns and product sustainability tags where data exists.

Suggested 10-column schema: order_id, order_date, product_category, unit_price, quantity, order_value, region, delivery_days, return_status, sustainability_label.

Example questions: Which categories dominate sales? How do returns vary by category? Are delivery times different between regions?

10 Space Observation and Satellite Data Trends

Explore accessible satellite or astronomical observation metadata (not image processing).

Suggested 10-column schema: observation_id, timestamp, satellite_or_sensor, region, observation_type, cloud_cover_pct, resolution_m, latitude, longitude, quality_flag.

Example questions: What coverage patterns emerge by region? How do cloud conditions affect observation quality? Which sensors contribute most records?

Data suitability note: Suggested columns are examples, not mandatory exact field names. Pick a real dataset whose available columns can legitimately support the question; never manufacture a missing measurement. Public datasets can be searched through portals such as Kaggle, Data.gov, World Bank, NASA Earthdata or city open-data portals. Record the precise source URL, owner, license and download date.

2 | Exactly what to do each week

WEEK 1 — Define the problem, acquire data and initialize GitHub

Weight: 20 marks

1. Select one topic; agree on the dataset, target users, 1 overall objective, 3 specific objectives and at least 5 answerable research questions.

2. Find an appropriate original dataset, verify legal/public use and document its URL, creator, license (if provided), collection period and limitations.

3. Create a GitHub repository with README.md, .gitignore, data/, notebooks/, visuals/ and report/ folders; add member names and assigned roles.

4. Write a beginner Python notebook containing variables, a list, a dictionary, a loop, a conditional and two small functions relevant to the investigation.

5. Load the dataset with Pandas and prove at least 15,000 rows and 10 meaningful columns using df.shape, df.columns and df.head(); save a screenshot of the output.

6. Commit the proposal, source information, first notebook and dataset schema; all members make at least one meaningful contribution.

Checkpoint: README + proposal (1–2 pages) + source citation + initial notebook + row/column proof + GitHub commits.

WEEK 2 — Inspect, validate and profile the dataset

Weight: 15 marks

1. Load data reliably with pd.read_csv() or an appropriate Pandas reader; make the notebook work from relative paths.

2. Show df.head(10), df.tail(5), df.shape, df.info(), df.dtypes and df.describe(include="all") where appropriate.

3. Create a data dictionary covering at least 10 columns: name, type, meaning, units/categories and expected missing values.

4. Check null counts and percentages, duplicate rows, invalid numeric ranges, inconsistent categories and date parsing; do not silently change source data.

5. Write at least two reusable functions such as validate_columns() and summarize_missingness(); give clear outputs.

6. Write five initial observations and three potential data-quality issues in Markdown, with evidence from notebook output.

Checkpoint: 02_exploration.ipynb + data dictionary + quality audit table + updated README and commits.

WEEK 3 — Clean the data and answer questions

Weight: 25 marks

1. Choose defensible treatments for missing values (retain, drop, fill) and document a reason for each choice.

2. Fix types and formats, normalize category text, handle duplicates, and flag outliers instead of automatically deleting them.

3. Save cleaned output to data/processed/; prove that the cleaned dataset still meets 15,000 rows and 10 useful columns (otherwise revise method or dataset).

4. Use filtering, sorting, groupby(), value_counts(), aggregations, mean/median/min/max and, where relevant, date-based comparisons.

5. Answer at least 5 project-specific analytical questions with a result table and 2–3 sentences of interpretation per question.

6. Create a before/after cleaning summary showing counts of nulls, duplicates, corrected fields and rows retained.

Checkpoint: 03_cleaning_analysis.ipynb + processed dataset + cleaning log + five answered questions + commits.

WEEK 4 — Build and explain Matplotlib visuals

Weight: 20 marks

1. Create at least 5 different meaningful visualizations: a comparison bar chart, time-series line chart, distribution histogram, category-share chart (pie or horizontal bar) and one additional suitable visual.

2. Choose a chart type appropriate to the variables; where pie charts are unsuitable, explain the alternative.

3. Add titles, units, axis labels, readable ticks, clear legends where needed, consistent formatting and figure captions.

4. Export all charts to visuals/ as PNG at readable resolution; keep plotting code in 04_visualization.ipynb.

5. Under every figure, add 3–4 sentences explaining the pattern, why it matters, and a limitation or caveat.

6. Create a 1-page findings summary with at least 3 data-backed conclusions and 2 practical recommendations.

Checkpoint: 04_visualization.ipynb + at least 5 PNGs + chart commentary + findings summary + commits.

WEEK 5 — Reproduce, document, submit and defend

Weight: 20 marks

1. Integrate notebooks or provide a clear numbered execution order; restart kernel and Run All without missing-file or variable errors.

2. Finalize README with project description, dataset provenance, installation commands, folder structure, execution steps, team contributions and key findings.

3. Write a 5–7-page PDF report: background, objectives, dataset and ethics, methods, cleaning, results, visualizations, recommendations, limitations and references.

4. Create 6–8 presentation slides and prepare a 10-minute group demonstration plus up to 5 minutes of questions.

5. Present GitHub commit history; every member must explain one code section, at least one result and their own contribution.

6. Publish the final commit/tag v1.0 and submit only the repository URL to Blackboard before the deadline assigned by the lecturer.

Checkpoint: complete public/access-granted repository + report PDF + presentation + final reproducibility demonstration.

3 | GitHub repository and submission protocol

Required layout (folder or file names may be equivalent but must be clear):

- README.md and requirements.txt (pandas, matplotlib and any necessary reader library)

- data/raw/ (original CSV if redistribution is permitted; otherwise source link and download instructions), data/processed/ (cleaned data or reproducible script if data is too large)
- notebooks/01_foundations.ipynb, 02_exploration.ipynb, 03_cleaning_analysis.ipynb, 04_visualization.ipynb
- visuals/ (exported images), report/final_report.pdf, presentation/final_slides.pdf or .pptx, docs/data_dictionary.md, docs/cleaning_log.md
- .gitignore excludes virtual environments, private information, API keys and large temporary files

GitHub evidence: At each weekly checkpoint, push the relevant files before the lecturer's deadline. Each teammate must have identifiable, substantive contributions across the five weeks. Empty or cosmetic commits are not acceptable. Use Git LFS or a documented external dataset link where GitHub size limits or permissions prevent uploading a 15,000+ row file. The analysis must remain reproducible.

Submission format: Group name, student names/IDs, selected project number, GitHub repository URL, GitHub final commit hash and brief contributions table entered in Blackboard. For private repositories, grant the lecturer access before submission.

4 | Marking rubric

Week	What earns the marks	Marks
1	Project scope/questions (5); credible dataset and 15k×10 validation (7); fundamentals, repository and member commits (8)	20
2	Data dictionary/profile (5); quality audit (6); reusable functions, initial observations and commits (4)	15
3	Documented cleaning decisions (8); valid processed dataset (5); five analytical answers (10); reproducible commits (2)	25
4	Five correct charts (10); labeling/readability (4); interpretation and findings (4); exported outputs/commits (2)	20
5	Executable repository and README (6); final report (6); presentation/live demonstration (5); individual defense/commit record (3)	20
TOTAL		100

5 | Quality checks before final submission

- The original and cleaned datasets each contain ≥15,000 rows and ≥10 meaningful columns; any reductions are justified and minimum requirements still met.
- All five analytical questions are answered with computed results rather than unsupported statements.
- All five visualizations have readable labels, interpretation and exported PNG files.
- The README contains dataset source and usage rights; a lecturer can recreate the analysis from repository instructions.
- No personal identifiers, secret tokens or unlawfully shared data are in the repository.
- Each team member can explain both the Python operations and what the results mean.

Academic integrity: Students may consult documentation or use coding assistance, but must identify external support and independently understand and defend submitted work. Fabricated empirical conclusions and copied work presented as original are not acceptable.